JULIO GARAVITO COLOMBIAN SCHOOL OF ENGINEERING



Assignment:

LAB 5 – Reverse Engineering

Teacher:

Ing. Daniel Vela

Members:

Ivan Santiago Forero Torres

Juan Sebastián Beltran Rodríguez

Nicolas Cuervo Naranjo

Systems Engineering

Bogotá D.C

24/2/26

Contents

INTRODUCTION. 3
Methodology. 3
Executive Summary. 4
Technical Summary. 4
Scope of Analysis. 4
Limitations of the Study. 5
Development: 6
Part 0: Download the binary: 6
Part 1: Initial recognition. 7
Part 2: Decompilation with Ghidra. 12
Part 3: Dynamic analysis congdb. 22
Part 4: Visual Flow Analysis with Cutter. 24
Conclusions: 30

INTRODUCTION.

This laboratory focuses on the application of Reverse Engineering methodologies on an unknown binary called passguess, obtained from the *crackmes.one* platform. The fundamental purpose is the analysis of a compiled program to decipher its internal logic, identify security mechanisms and recover critical information – in this case, an access password – without having the original source code or previous technical documentation.

This process is vital in the field of cybersecurity for auditing tasks, malware analysis and search for vulnerabilities in proprietary software. The study is divided into three dimensions of analysis:

1. Basic Static Analysis: Identification of file properties using command-line tools such as file, strings, and objdump. The binary was determined to be a 64-bit ELF executable, dynamically linked and, crucially, not stripped, allowing the original function names (symbols) to be preserved for ease of study.
2. Advanced Static Analysis (Decompilation): Use of the Ghidra tool to reverse engineer the machine code into C pseudocode. This step allows you to identify the strcmp string comparison function and locate the literal constant "7Wtyr", which acts as the access key to the system.
3. Dynamic Analysis: Verification of program behavior at runtime using the GDB debugger. This phase confirms the processor's interaction with memory, observing how the RDI and RSI registers load the arguments of the entered password and the actual password before the logical branch.

Finally, the analysis is supported by a visual representation of the Flow Control Graph (GFC), identifying the decision node at address 001011ec. This graph reveals the dichotomous structure of the binary, where a successful result of the comparison directs execution towards the [OK] Passowrd Found message, while any other input leads to the error block. The success of this lab lies not only in obtaining the key, but in validating the complete flow from data input to program output.

Methodology.

For this analysis, a black box analysis approach was implemented structured in four progressive phases:

1. Initial Recognition (Basic Static Analysis): Use of system commands (file, strings, objdump) to identify binary architecture (x86-64), linked libraries, and suspicious text strings in the data section.
2. Advanced Decompilation (Ghidra): Reconstruction of the logical flow in C pseudocode to identify critical comparison functions (strcmp) and renaming of variables to facilitate the understanding of the program's logic.
3. Dynamic Analysis (GDB): Real-time execution interception using *breakpoints* to inspect the contents of CPU registers ($rdi, $rsi) and the return value in $rax during password comparison.
4. Visual Flow Analysis (Cutter/Ghidra Graph): Flow Control Graph (CFG) modeling to visualize decision blocks and success and failure execution paths.

Executive Summary.

The analysis determined that the Passguess binary has zero critical security, as it stores the access password statically and unencrypted within the executable. The password "7Wtyr" was successfully identified after inspecting the strings and flow of the main function. Further testing confirmed that entering this string results in a successful [OK] Password Found message, validating that the authentication mechanism relies entirely on a simple plaintext comparison in memory.

Technical Summary.

Technically, the binary is a 64-bit ELF executable, with stack canary (__stack_chk_fail) protection and PIE enabled. The authentication logic resides in the main function, which uses scanf in a %255s format to receive user input. This input is compared by the strcmp function against the constant stored in the memory address loaded into the %rsi register.

- Decision Point: The instruction at offset 11ec (test %eax, %eax) followed by jne 1201 defines the destination of the flow.
- Dynamic Analysis: The $rax register returns 0 when there is an exact match, allowing the program to advance to the block that prints the success message.

Scope of Analysis.

- **Target Binary:** Passguess executable retrieved from crackmes.one.
- **Environment:** Linux operating system (Kali Linux) with x86-64 architecture.

- **Specific Objectives:** Identification of the architecture, extraction of the password, mapping of the control flow and execution of a dynamic bypass by debugging.

Limitations of the Study.

- **Absence of Obfuscation:** The binary does not have packing or *anti-debugging* techniques, which facilitated direct analysis.
- **Local Analysis:** The study is limited to the provided version of the binary and does not contemplate external or network vulnerabilities, since the program is autonomous and runs locally.
- **Available Symbols:** The binary was not "*stripped*", which allowed the original names of the functions to be seen as main, significantly simplifying the initial recognition phase.

Development:

In this lab, you'll analyze a binary that prompts the user for a password. Your goal is to find the correct password without accessing the source code, using only reverse engineering tools and techniques.

Part 0: Download the binary:

1. Download crackme from crackmes.one. You will receive a .zip file.

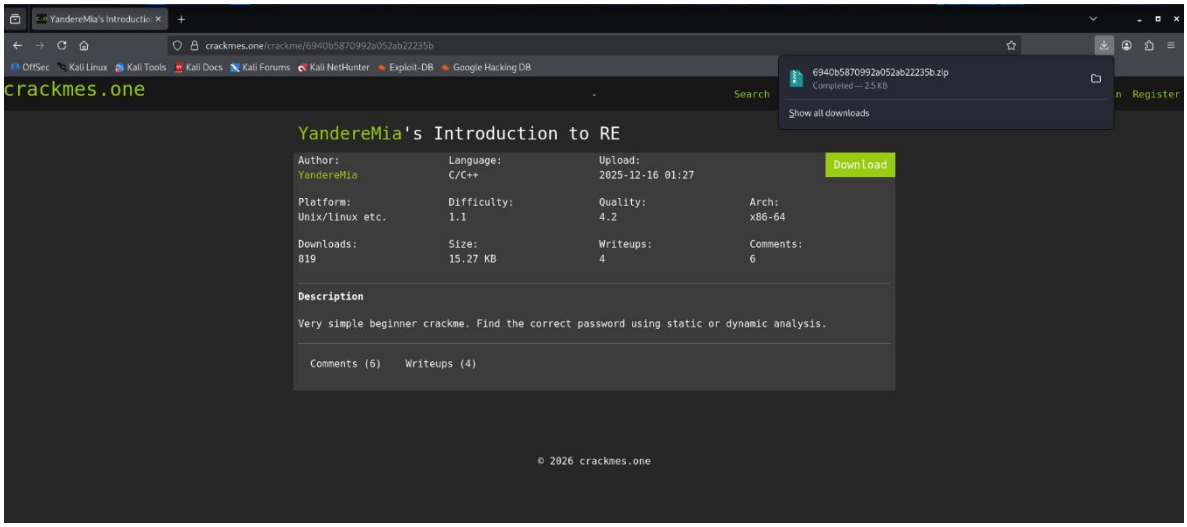


Image 1. Downloading Crackme from https://crackmes.one/lasts/1

2. Unzip it. The zip file is password protected crackmes.one:

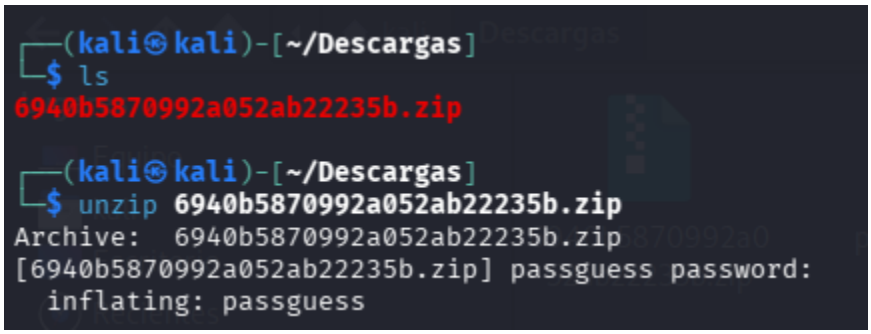


Image 2. Unzip the file .zip and password "crackmes.one"

3. Make the binary executable:

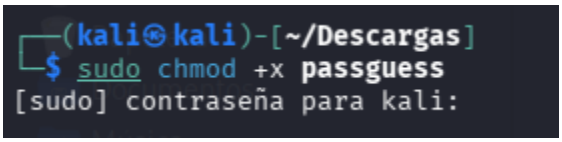


Image 3. Binary with execute permissions.

You should now have the binary passguess ready to analyze.

Part 1: Initial recognition.

Before opening any advanced tool, gather some basic information about the binary.

1. **Determine the file type.** Run the following command and note if the binary is 32-bit or 64-bit, statically or dynamically linked, and if the symbols have been removed:



Image 4. Running the "file passguess" command

Findings:

- **ELF 64-bit LSB executable foot, x86-64:** 64-bit binary.
- **PIE (Position Independent Executable):** The memory addresses change each time the program is executed.
- **Dynamically linked**: The program uses external libraries.
- **Not stripped**: The binary retains the symbols, this means that when we analyze it, we will see the real names of the functions as main.

2. **Extracts readable strings.** Many binaries contain plaintext strings (prompts, error messages, encoded values). Look for them:

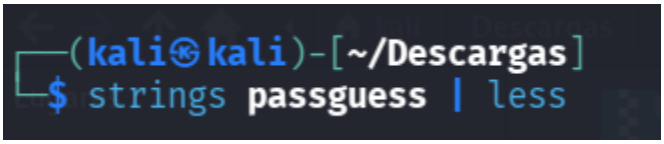
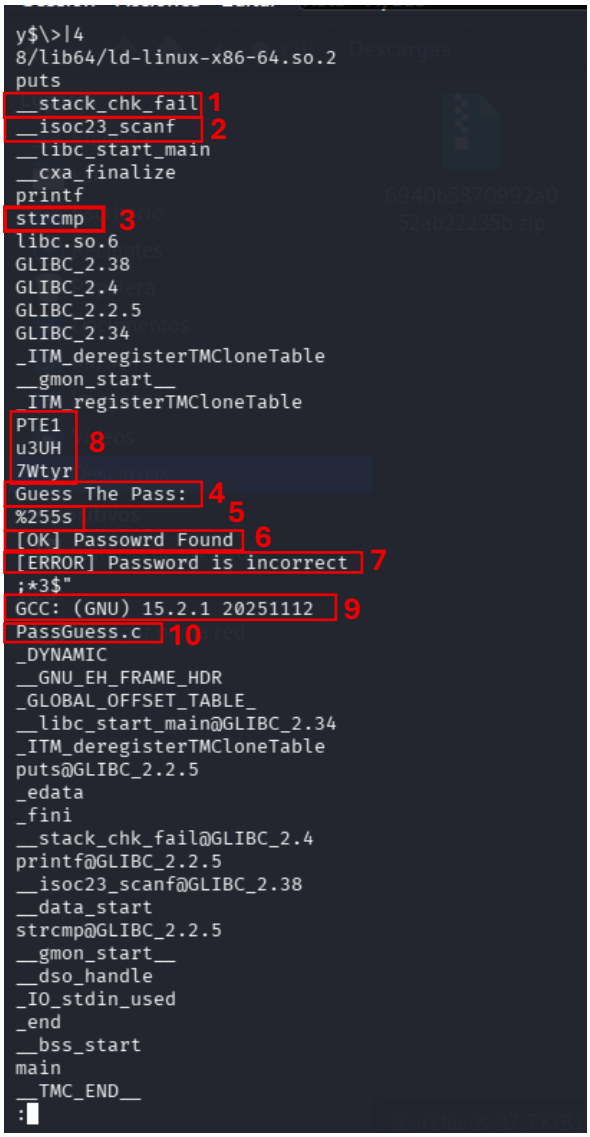
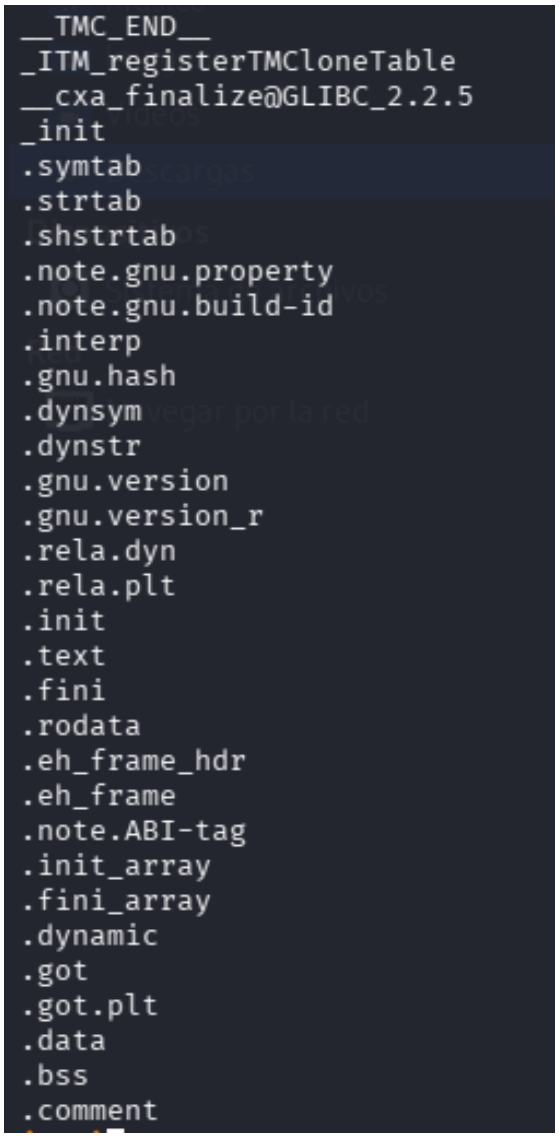


Image 5. Running the command "strings passguess | them"





Images 6 and 7. Output elements when running the command "String passguess | less"

Findings:

1. **__stack_chk_fail:** This is a security function, if there is an attack to fill the memory this function detects it and closes the program.
2. **__isoc23_scanf:** Function that reads the keyboard.
3. **Strcmp:** String Compare function, if used it is because somewhere there must be a password stored.
4. **Guess The Pass:** The initial message.
5. **%255s:** This is the format used by scanf. Indicates that the program accepts a word of up to 255 characters.

6. **[OK] Passowrd Found**: The Message of Success.
7. **[ERROR] Password is incorrect**: The error message.
8. **PTE1, u3UH, 7Wtyr:** Nonsensical strings, main suspects of being the key.
9. **GCC: (GNU) 15.2.1 20251112:** Tells us the exact version of the compiler and the date.
10. **PassGuess.c**: By having the name of the file we are analyzing we can conclude that it is the file where the source code is and the .c indicates that it was written in c.

3. **Inspect the disassembly to identify key functions.** Get a teardown list and look for known feature names:

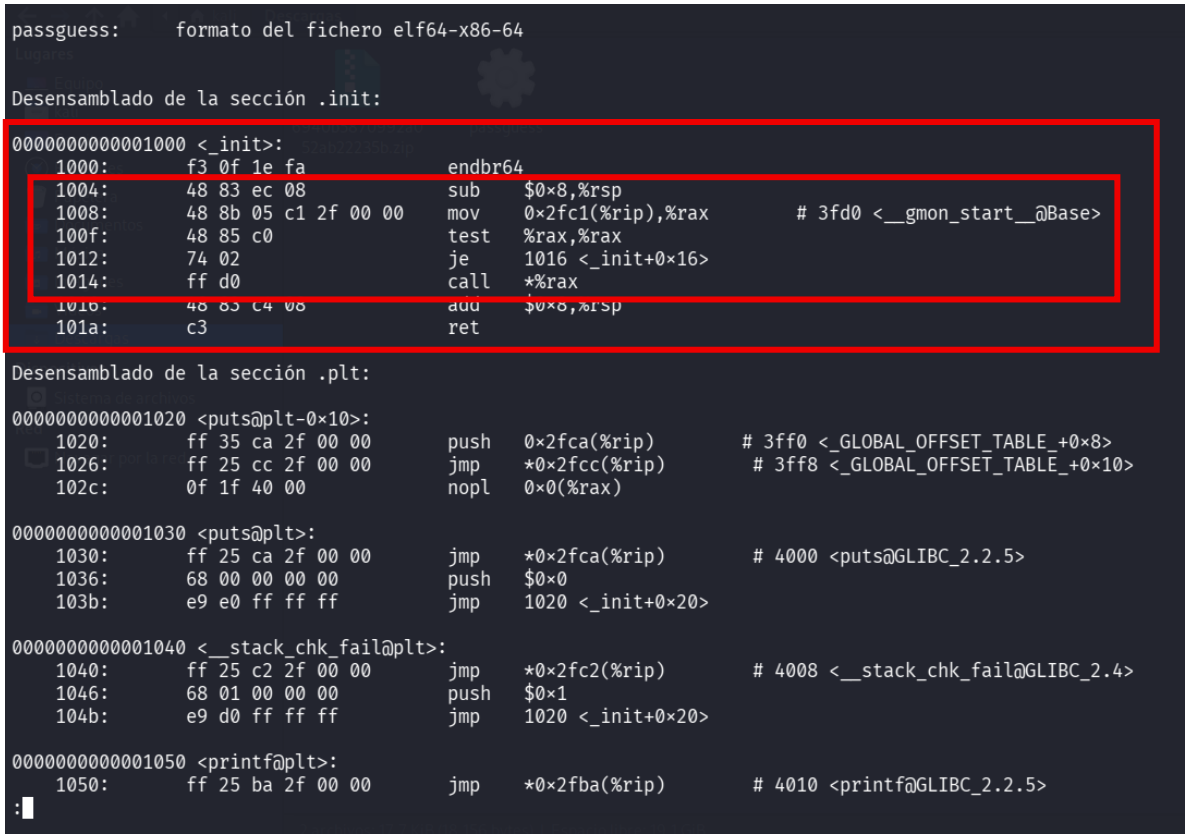


Image 9. Output of the command "objdump -d passguess | less"

- First this piece of code is executed which is like the one that on a large scale gives us the "Welcome", in this part for now we do not have any kind of logic related to passwords.
- Endbr64 is a security statement to prevent flow redirection attacks and starts monitoring the application.

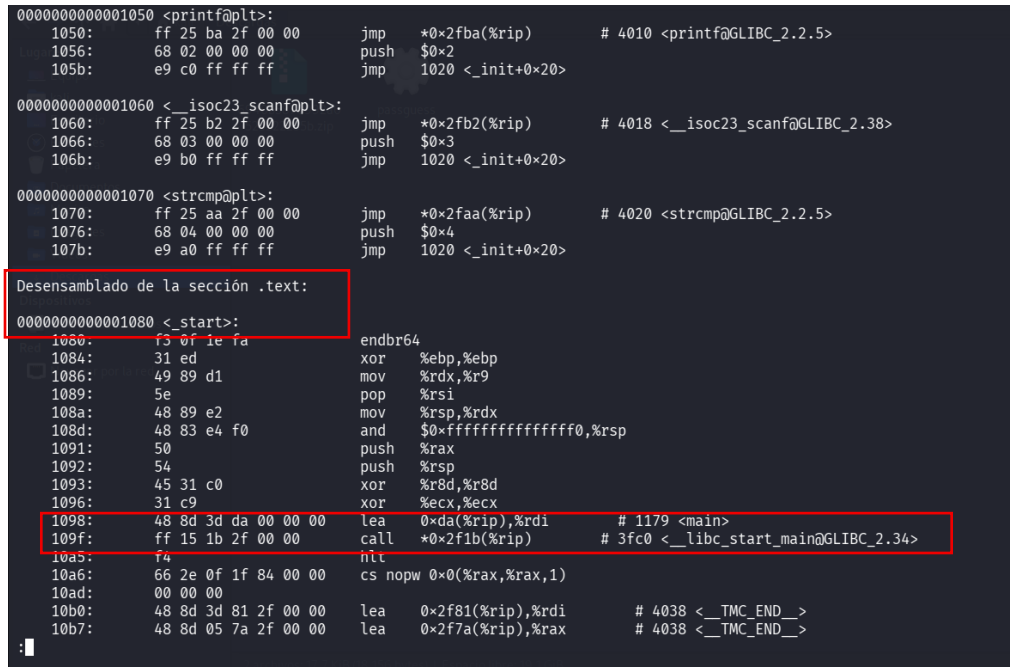


Image 10. Output of the command "objdump -d passguess | less"

- Our binary doesn't exactly start with a .main, first it calls the _start function and on line 1098 it uses lea (Load Effective Address) to load main.

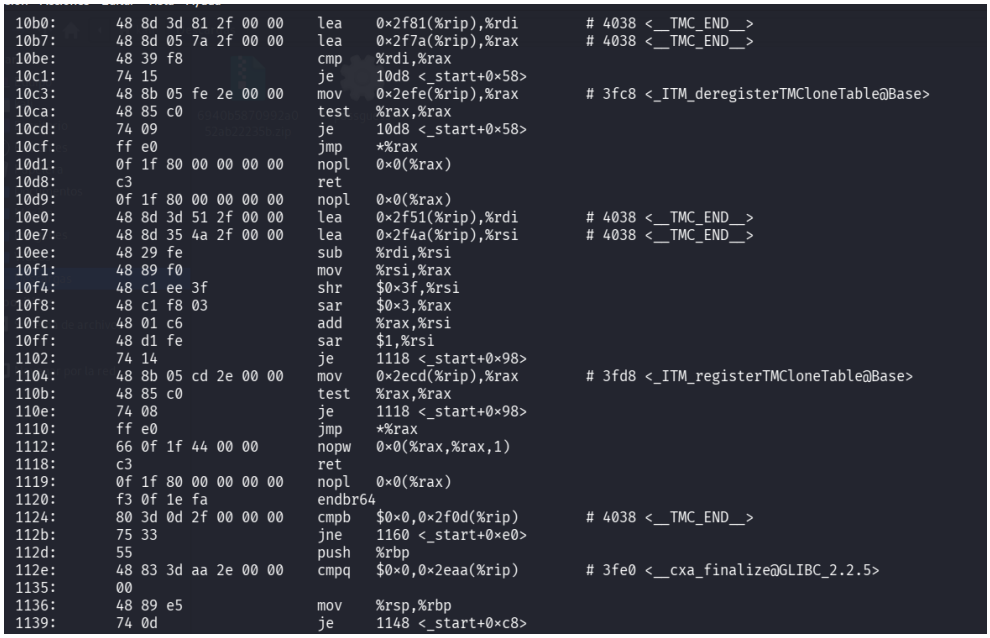
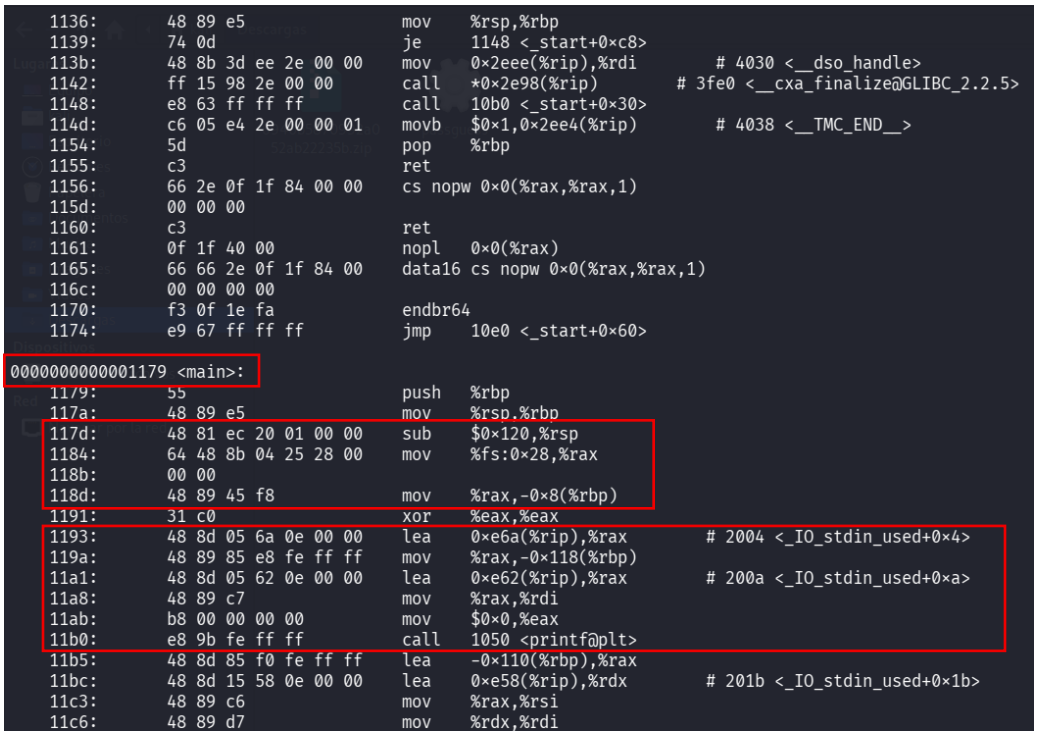


Image 11. Output of the command "objdump -d passguess | less"



```
1136:       48 89 e5                mov    %rsp,%rbp
1139:       74 0d                   je     1148 <_start+0x8>
113b:       48 8b 3d ee 2e 00 00    mov    0x2eee(%rip),%rdi        # 4030 <__dso_handle>
1142:       ff 15 98 2e 00 00       call   *0x2e98(%rip)        # 3fe0 <__cxa_finalize@GLIBC_2.2.5>
1148:       e8 63 ff ff ff          call   10b0 <_start+0x30>
114d:       c6 05 e4 2e 00 00 01    movb   $0x1,0x2ee4(%rip)        # 4038 <__TMC_END__>
1154:       5d                      pop    %rbp
1155:       c3                      ret
1156:       66 2e 0f 1f 84 00 00    cs nopw 0x0(%rax,%rax,1)
115d:       00 00 00
1160:       c3                      ret
1161:       0f 1f 40 00             nopl   0x0(%rax)
1165:       66 66 2e 0f 1f 84 00    data16 cs nopw 0x0(%rax,%rax,1)
116c:       00 00 00 00
1170:       f3 0f 1e fa             endbr64
1174:       e9 67 ff ff ff          jmp    10e0 <_start+0x60>

0000000000001179 <main>:
1179:       55                      push   %rbp
117a:       48 89 e5                mov    %rsp,%rbp
117d:       48 81 ec 20 01 00 00    sub    $0x120,%rsp
1184:       64 48 8b 04 25 28 00    mov    %fs:0x28,%rax
118b:       00 00
118d:       48 89 45 f8             mov    %rax,-0x8(%rbp)
1191:       31 c0                   xor    %eax,%eax
1193:       48 8d 05 6a 0e 00 00    lea    0xe6a(%rip),%rax        # 2004 <_IO_stdin_used+0x4>
119a:       48 89 85 e8 fe ff ff    mov    %rax,-0x118(%rbp)
11a1:       48 8d 05 62 0e 00 00    lea    0xe62(%rip),%rax        # 200a <_IO_stdin_used+0xa>
11a8:       48 89 c7                mov    %rax,%rdi
11ab:       b8 00 00 00 00          mov    $0x0,%eax
11b0:       e8 9b fe ff ff          call   1050 <printf@plt>
11b5:       48 8d 85 f0 fe ff ff    lea    -0x110(%rbp),%rax
11bc:       48 8d 15 58 0e 00 00    lea    0xe58(%rip),%rdx        # 201b <_IO_stdin_used+0x1b>
11c3:       48 89 c6                mov    %rax,%rsi
11c6:       48 89 d7                mov    %rdx,%rdi
```

Image 12. Output of the command "objdump -d passguess | less"

- We start with the central part of our program, the main function.
- The next part is the security section of our main, there you can see that they take a secret value "%fs:0x28" and place it at the end of main, if the memory overflows this value is deleted and the program will notice, thus causing it to terminate.
- In 1193 it loads the 2004 section where our real password can possibly be and saves it in -0x118(%rbp), in part 11a1 it loads the 200th section which is the text "Guess the pass: ".

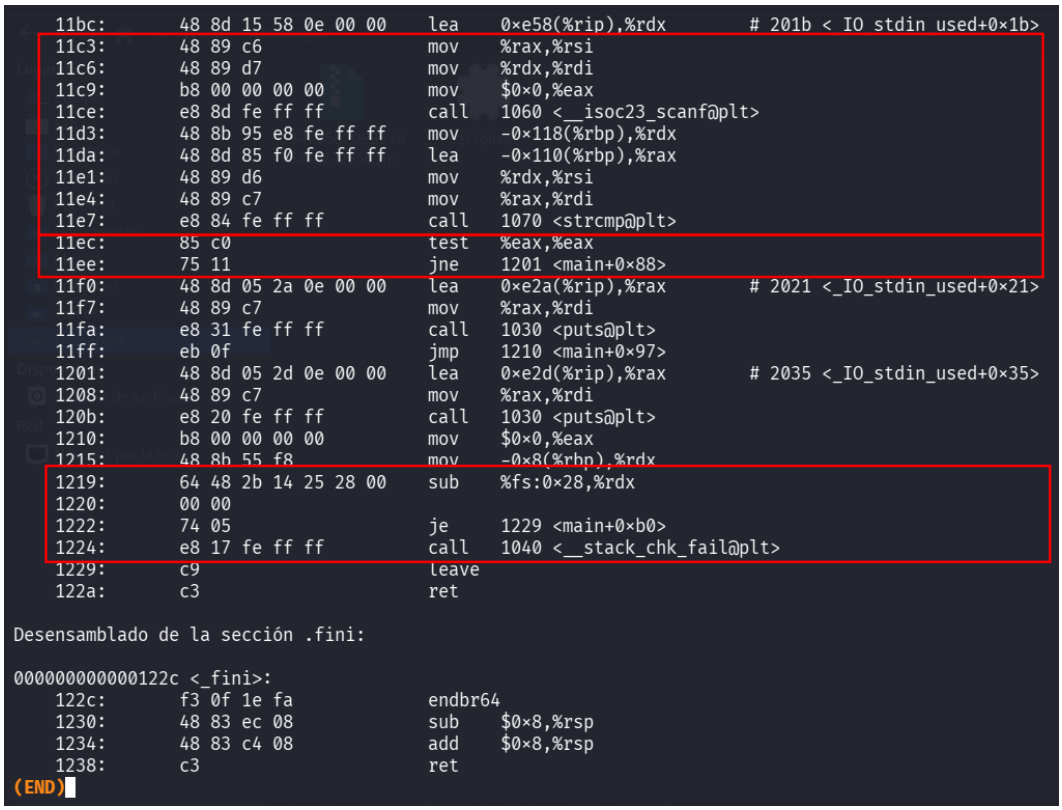


Image 13. Output of the command "objdump -d passguess | less"

- In section 11ce we can see that there is a scanf so we can know that our key is being saved.
- 11d3 retrieves the actual key and puts it in %rdx.
- 11th loads our answer in %rax.
- %rdi our answer and %rsi is the actual key, there the comparison is executed.
- In section 11ec and 11ee is where the comparison of the passwords is made %eax test, %eax checks if it is zero, if jne (Jump if Not Equal) if they are NOT equal (result is not 0), jumps to address 1201, if it does not jump, it follows 11f0.
- The final part corresponds to the security check, if the value on the stack is different from the original, the program calls __stack_chk_fail and closes to prevent an exploit from taking control.

Part 2: Decompilation with Ghidra.

Ghidra can reconstruct rough C source code (pseudocode) from a binary, which is much easier to read than the raw assembly.

1. **Open Ghidra and create a new project** (File → New Project → Unshared Project). Give it a name.

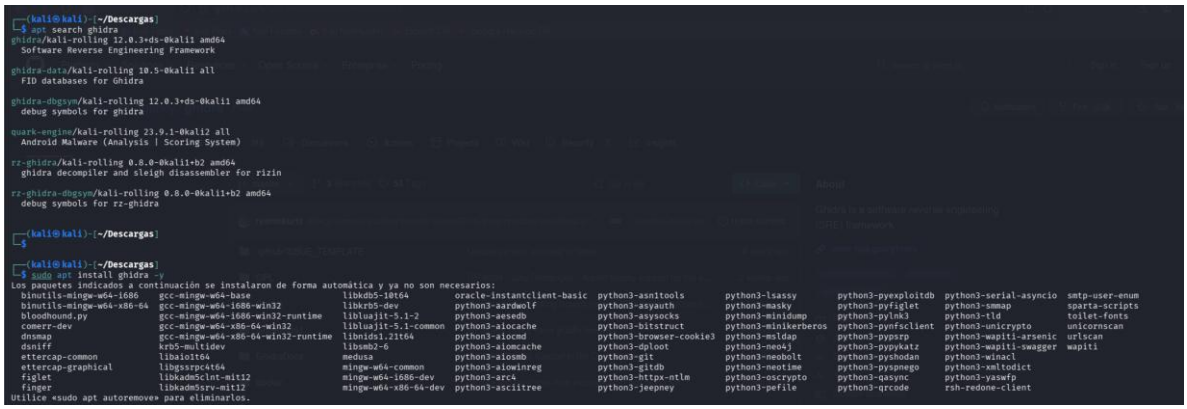


Image 14. Installation of ghidra on our Linux machine.

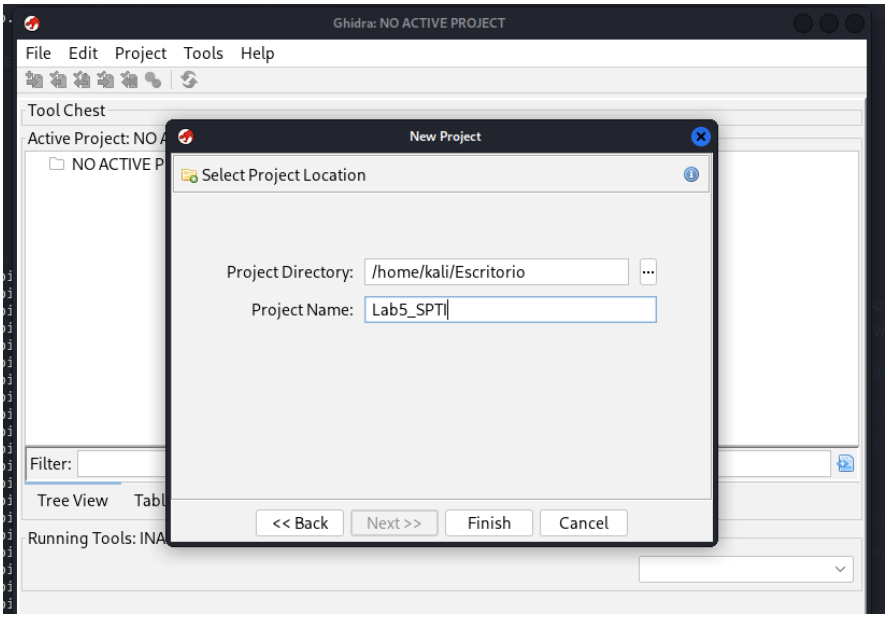


Image 15. Creation of the project under the name "Lab5_SPTI"

2. **Import the binary.** Go to File → Import File and select passguess. Ghidra will automatically detect the format and architecture. Click OK.

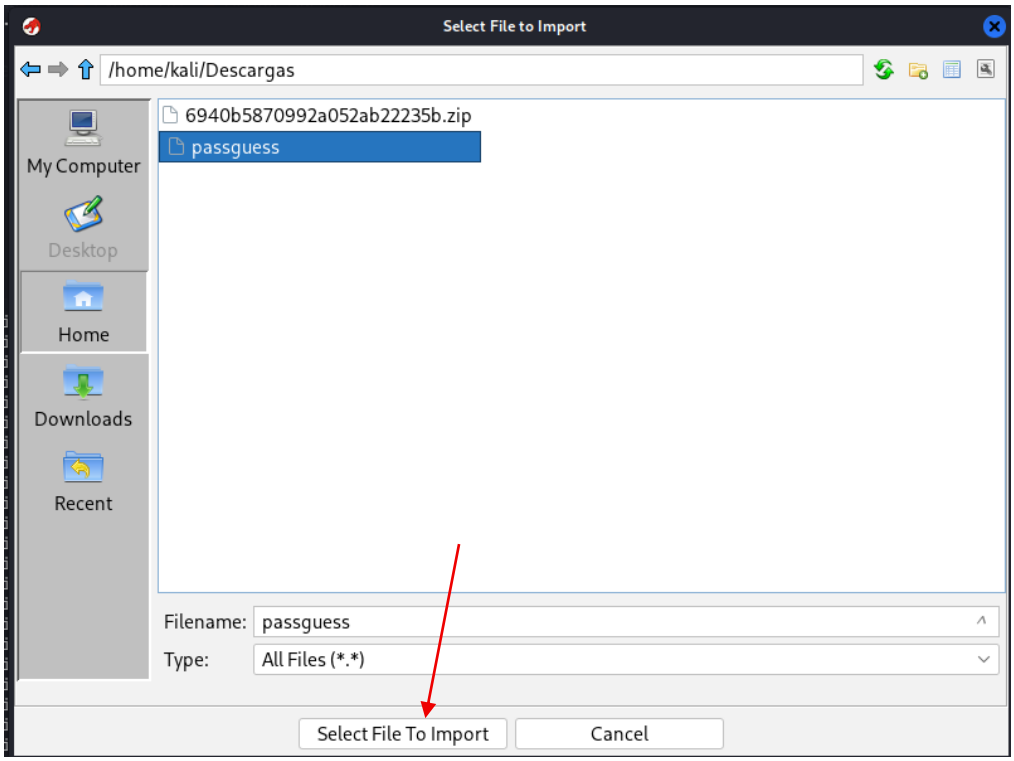


Image 16. Import of our file.

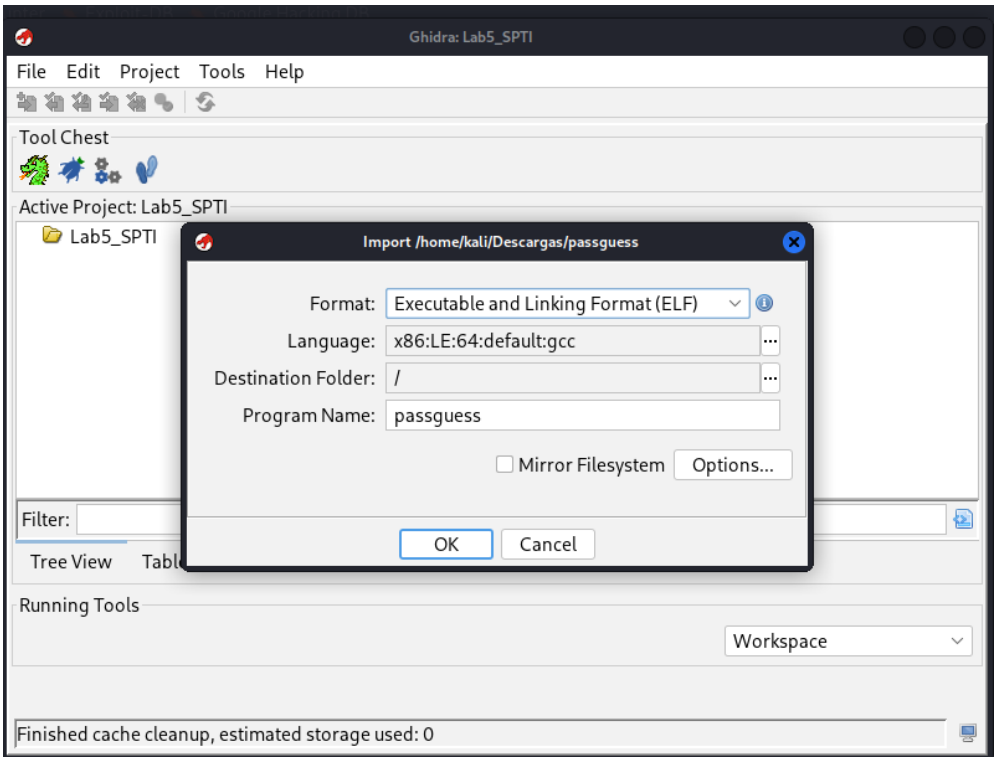


Image 17. Ghidra automatically recognizes the file structure

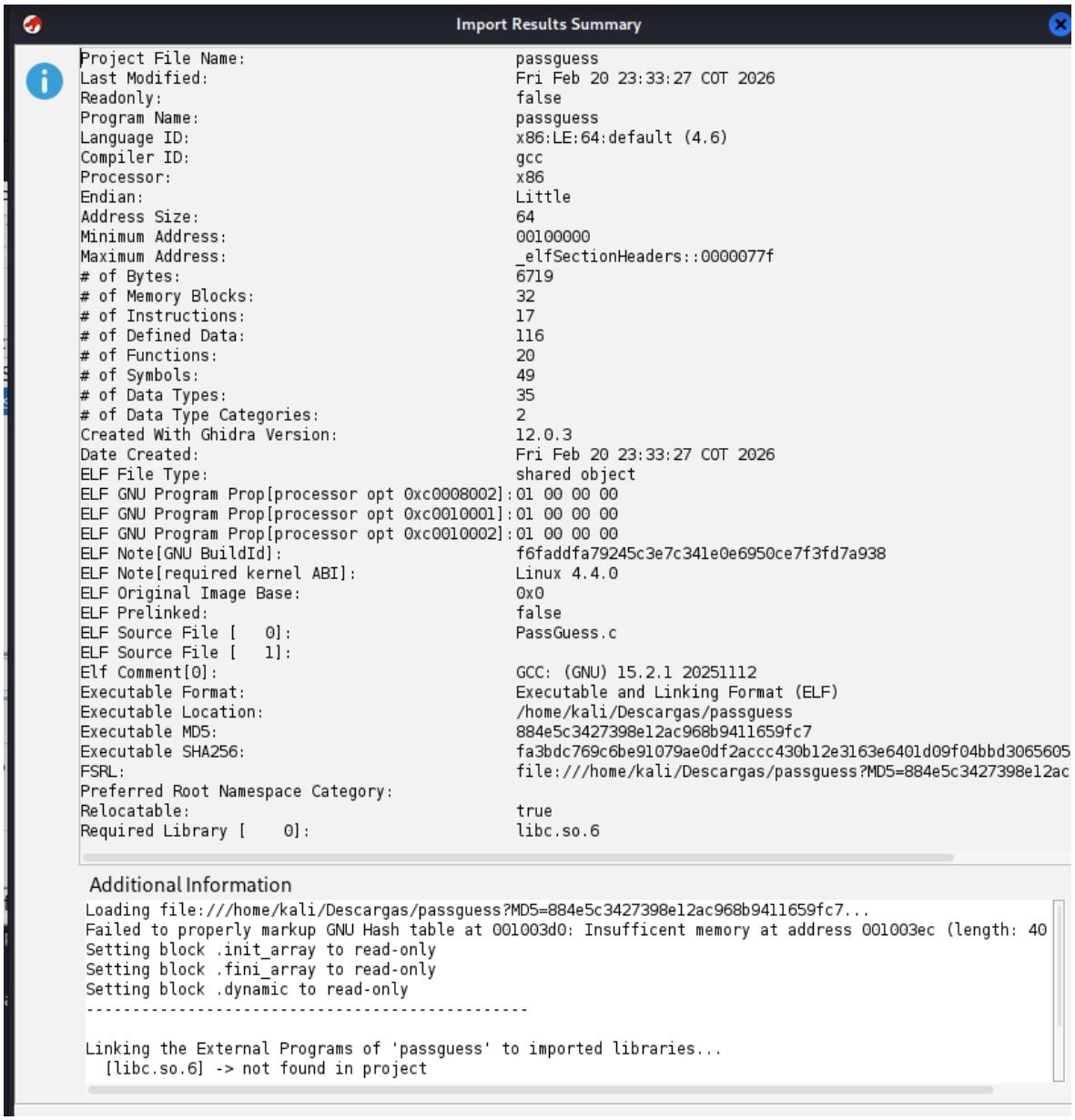


Image 18. Summary of the findings of the imported file.

3. **Run the automatic scan.** When the message "The program has not been scanned. Do you want to scan it now?", click "Yes" and accept the default parsers. Wait for the scan to finish (progress bar in the bottom right corner).

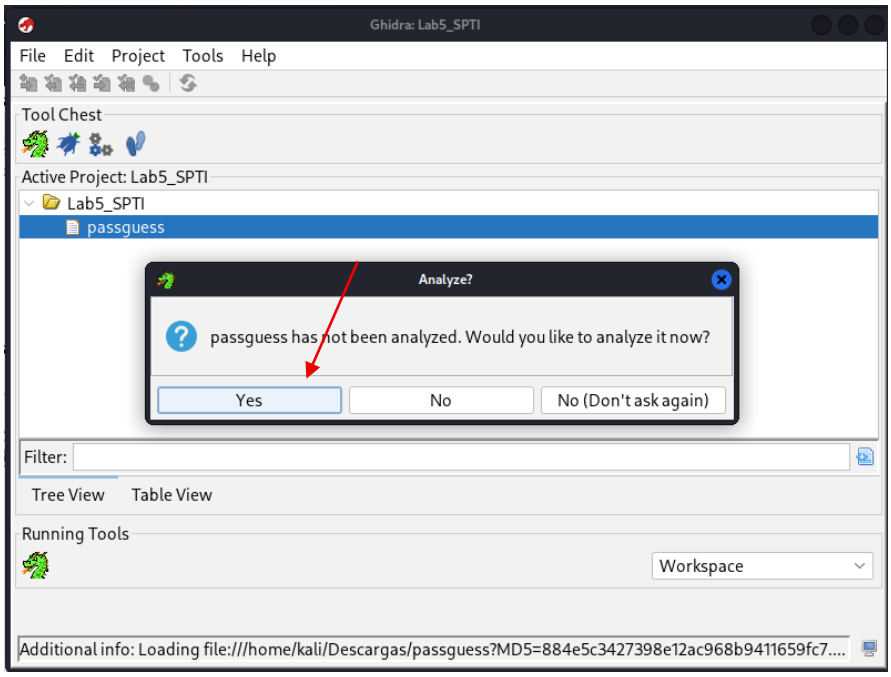


Image 19. Acceptance of automatic analysis in Ghidra.

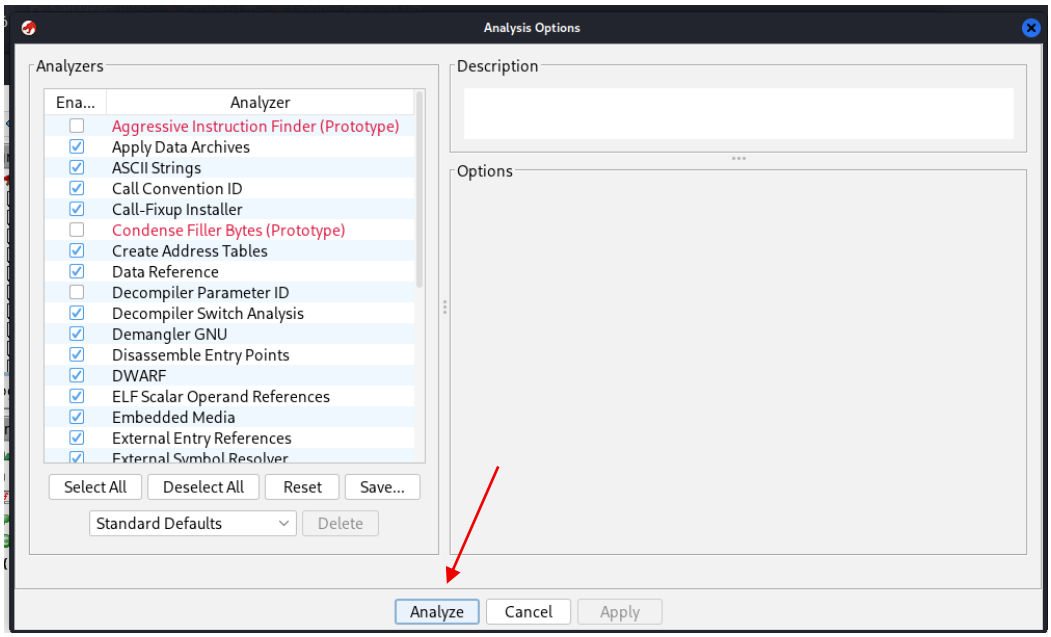


Image 20. We leave all default analyses and analyze.

4. **Go to main.** In the **Symbol Tree** pane , on the left, expand Functions and click Main. The **Listing panel** will show the disassembly, and the Decompile panel will show the pseudocode.

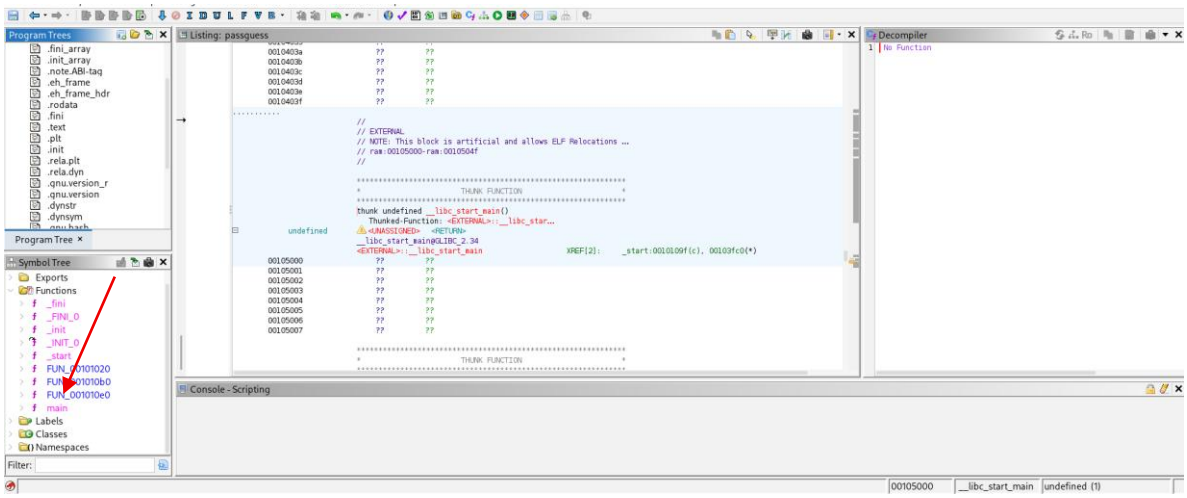


Image 21. We leave all default analyses and analyze.

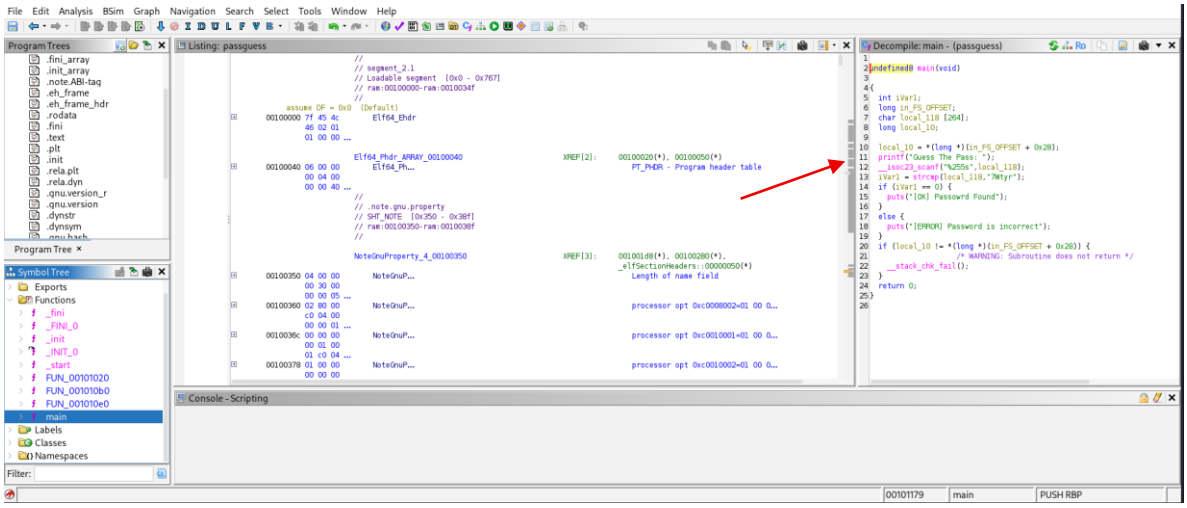


Image 22.Display of the code with its respective variables.

5. **Read the pseudocode carefully.** You should see something similar to this structure:
    - A local buffer (array) is declared on the stack (for example, local_118).
    - printfPrint a message requesting information.

- scanfRead the user's input in the buffer.
- strcmpCompares the buffer to an encoded string.
- Prints if/elsea message of success or failure.
- You may also see a __stack_chk_fail()called near the end: this is an automatic check of the stack inserted by the compiler for buffer overflows. You can ignore it in this exercise.

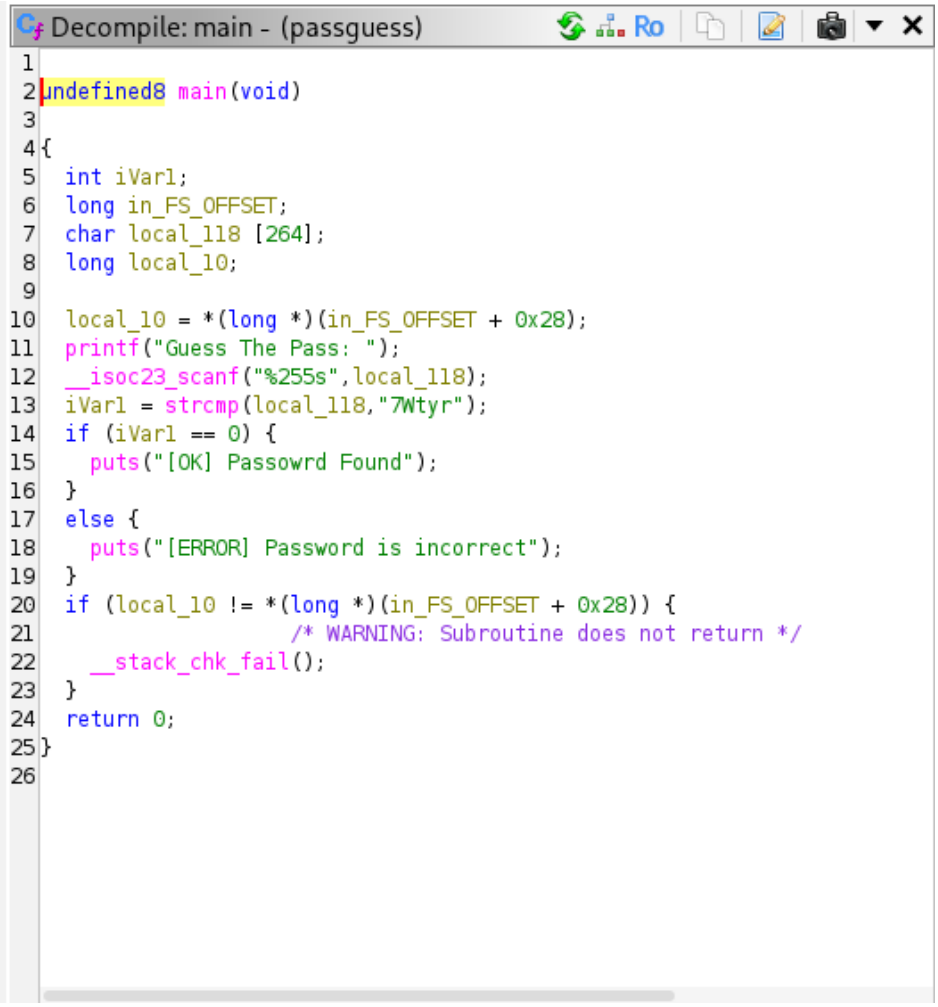


Image 23. Display of the code with its respective variables (No modification).

6. **Rename variables for clarity.** Ghidra generates placeholder names such as local_118o iVar1. You can rename them to make the pseudocode easier to read: right-click on a variable name and select Rename Variable (or press L). For example:
   - Rename the buffer (for example, local_118,) auser_input.

- Rename the strcmp return value (for example, iVar1,) apassword_match.

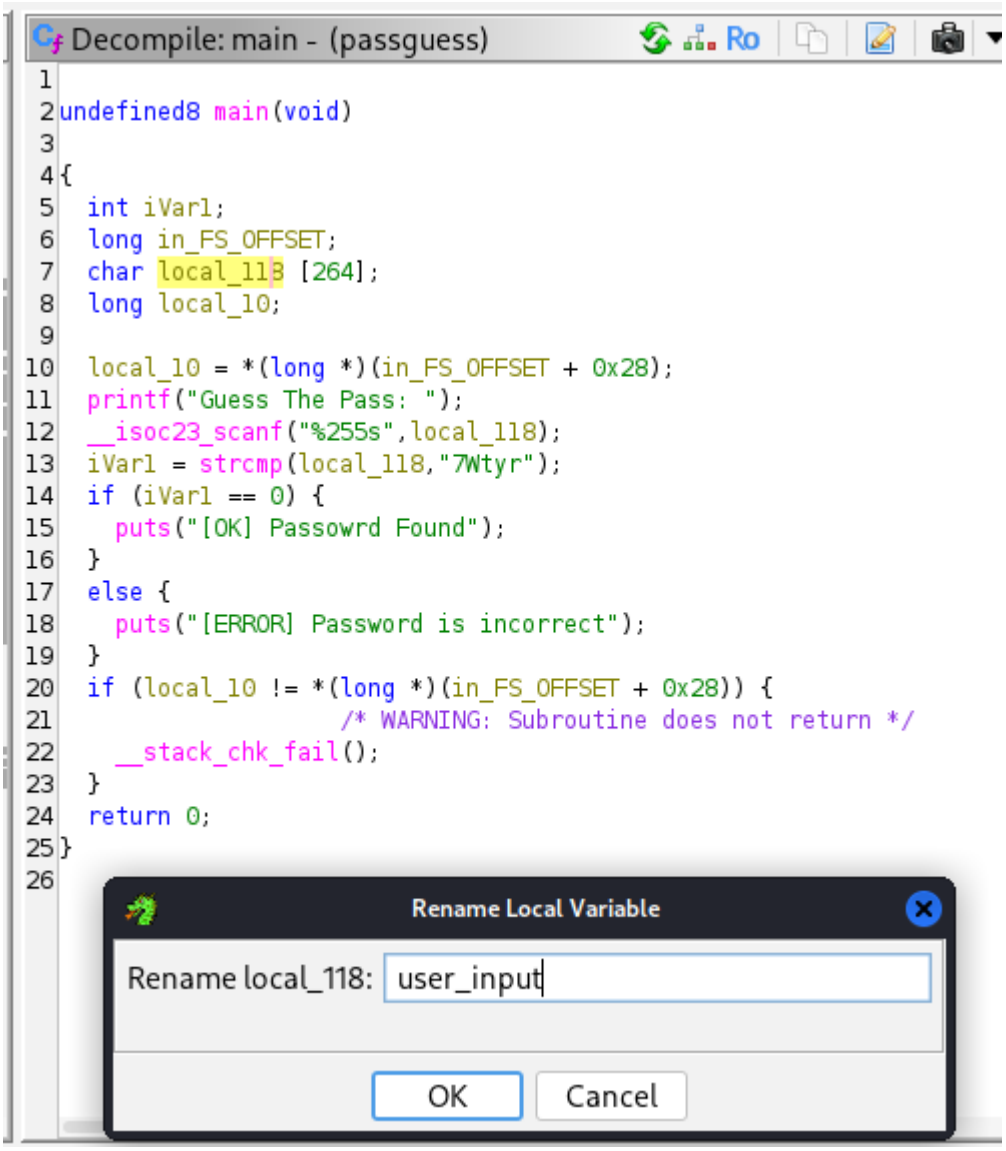


Image 24. Change of variable by the user's input for identification.

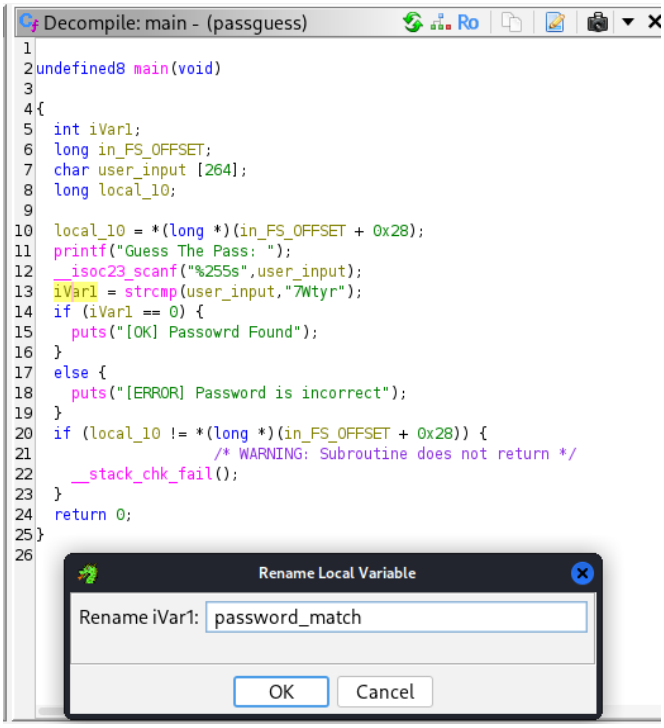


Image 25. Change of variable for the password of our system.

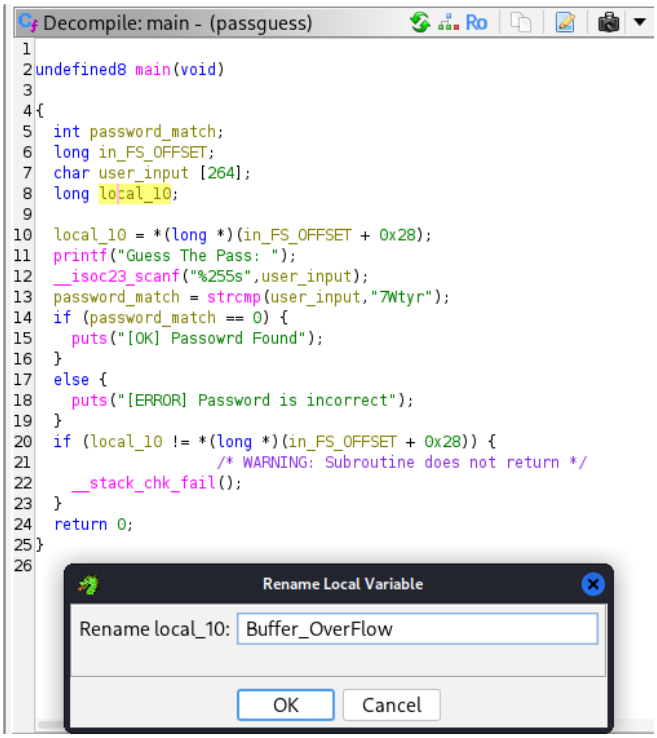


Image 26. Change of variable by buffer safety measure.

7. Identify the comparison. Notice the strcmpcalled. He accepts two arguments:
    - The buffer that contains the user's input.
    - A literal string: the encrypted password.
8. Extract the password. The second argument strcmp is the correct password. Write it down.

Decompile: main - (passguess)

```
1
2 undefined8 main(void)
3
4 {
5   int password_match;
6   long in_FS_OFFSET;
7   char user_input [264];
8   long Buffer_OverFlow;
9
10  Buffer_OverFlow = *(long *)(in_FS_OFFSET + 0x28);
11  printf("Guess The Pass: ");
12  __isoc23_scanf("%255s",user_input);
13  password_match = strcmp(user_input,"7Wtyr");
14  if (password_match == 0) {
15    puts("[OK] Passowrd Found");
16  }
17  else {
18    puts("[ERROR] Password is incorrect");
19  }
20  if (Buffer_OverFlow != *(long *)(in_FS_OFFSET + 0x28)) {
21                    /* WARNING: Subroutine does not return */
22    __stack_chk_fail();
23  }
24  return 0;
25 }
```

Image 27. Password display.

9. Verify your finding. Run the binary file and enter the password you found:



Image 28. Password checking.

Part 3: Dynamic analysis congdb.

Even if you already know the password, use the GDB password to dynamically confirm the search and practice debugging skills.

1. **Start the debugger:**



Image 29. We start debugger using "gdb ./passguess"



Image 30. GDB console display.

2. **Break Main and Run:**

```
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./passguess...
(No debugging symbols found in ./passguess)
(gdb) break main
Breakpoint 1 at 0x1184
(gdb) run
Starting program: /home/kali/Descargas/passguess
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x0000555555555184 in main ()
(gdb)
```

Image 31. Critical path visualization and conditional binary jumps

3. **Disassemble main to find the called strcmp.** Since the binary has no debug symbols, we can't move forward line by line with next. Instead, we'll find the exact address of the called strcmp and set a precise breakpoint:

```
(gdb) disassemble main
Dump of assembler code for function main:
   0x0000555555555179 <+0>:     push   %rbp
   0x000055555555517a <+1>:     mov    %rsp,%rbp
   0x000055555555517d <+4>:     sub    $0x120,%rsp
=> 0x0000555555555184 <+11>:    mov    %fs:0x28,%rax
   0x000055555555518d <+20>:    mov    %rax,-0x8(%rbp)
   0x0000555555555191 <+24>:    xor    %eax,%eax
   0x0000555555555193 <+26>:    lea    0xe6a(%rip),%rax        # 0x555555556004
   0x000055555555519a <+33>:    mov    %rax,-0x118(%rbp)
   0x00005555555551a1 <+40>:    lea    0xe62(%rip),%rax        # 0x55555555600a
   0x00005555555551a8 <+47>:    mov    %rax,%rdi
   0x00005555555551ab <+50>:    mov    $0x0,%eax
   0x00005555555551b0 <+55>:    call   0x555555555050 <printf@plt>
   0x00005555555551b5 <+60>:    lea    -0x110(%rbp),%rax
   0x00005555555551bc <+67>:    lea    0xe58(%rip),%rdx        # 0x55555555601b
   0x00005555555551c3 <+74>:    mov    %rax,%rsi
   0x00005555555551c6 <+77>:    mov    %rdx,%rdi
   0x00005555555551c9 <+80>:    mov    $0x0,%eax
   0x00005555555551ce <+85>:    call   0x555555555060 <__isoc23_scanf@plt>
   0x00005555555551d3 <+90>:    mov    -0x118(%rbp),%rdx
   0x00005555555551da <+97>:    lea    -0x110(%rbp),%rax
   0x00005555555551e1 <+104>:   mov    %rdx,%rsi
   0x00005555555551e4 <+107>:   mov    %rax,%rdi
   0x00005555555551e7 <+110>:   call   0x555555555070 <strcmp@plt>
   0x00005555555551ec <+115>:   test   %eax,%eax
   0x00005555555551ee <+117>:   jne    0x555555555201 <main+136>
   0x00005555555551f0 <+119>:   lea    0xe2a(%rip),%rax        # 0x555555556021
   0x00005555555551f7 <+126>:   mov    %rax,%rdi
   0x00005555555551fa <+129>:   call   0x555555555030 <puts@plt>
   0x00005555555551ff <+134>:   jmp    0x555555555210 <main+151>
   0x0000555555555201 <+136>:   lea    0xe2d(%rip),%rax        # 0x555555556035
   0x0000555555555208 <+143>:   mov    %rax,%rdi
   0x000055555555520b <+146>:   call   0x555555555030 <puts@plt>
   0x0000555555555210 <+151>:   mov    $0x0,%eax
   0x0000555555555215 <+156>:   mov    -0x8(%rbp),%rdx
   0x0000555555555219 <+160>:   sub    %fs:0x28,%rdx
   0x0000555555555222 <+169>:   je     0x555555555229 <main+176>
   0x0000555555555224 <+171>:   call   0x555555555040 <__stack_chk_fail@plt>
   0x0000555555555229 <+176>:   leave
   0x000055555555522a <+177>:   ret
```

Image 32. identified that the program uses the strcmp function to validate user input against the constant string "7Wtyr".

4. **Set a breakpoint in the strcmp call address and continue.** Replace the address below with the one you found. This ensures that only password matching is interrupted, not internal calls to the strcmp library:

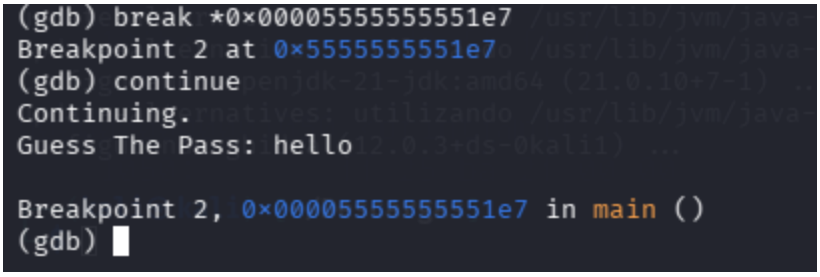


Image 33. We verify breakpoint.

5. **Inspect the arguments**. In x86-64, function arguments are passed in registers: rdione contains the first argument and rsiel another contains the second. Examine them as strings:

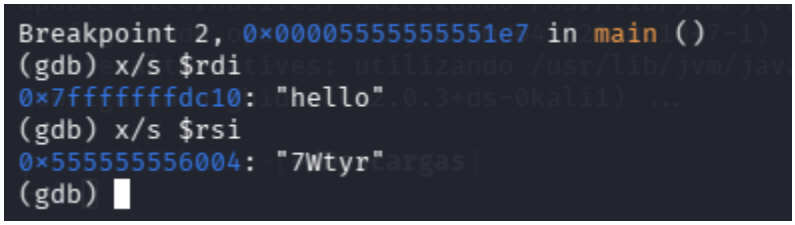


Image 34. We perform the chain comparison.

6. **Inspect the return value.** Enter strcmpy then use finish to complete it. The return value will be rax:



Image 35. Implementation of stepi and finish commands.

Part 4: Visual Flow Analysis with Cutter.

Cutter (or Ghidra's graphical view) provides a visual representation of the program's control flow, making it easy to see decision points.

1. **Open the binary in Cutter** (or use the Ghidra Function Graph view: Window → Function Graph).

2. **Navigate main** using the feature list or search bar.
3. **Switch to the chart view**. You should see a flowchart with blocks connected by arrows:
    - A block that calls scanf to read the input.
    - A block that calls strcmp to compare strings.
    - A conditional branch that splits into two paths:
        - Success Path → prints the message "OK".
        - Error path → the message "ERROR" prints.
4. **Take a screenshot of** the chart. Write it down to show:
    - Where user input is read.
    - Where the comparison occurs.
    - Which branch leads to success and which to failure?

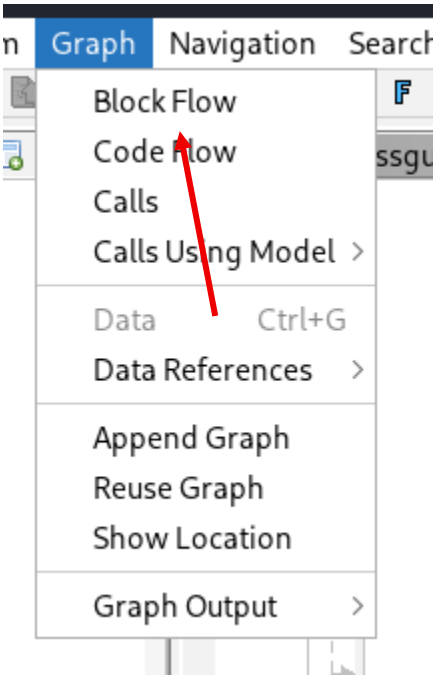


Image 36. Select the Code Flow option to generate the diagram.

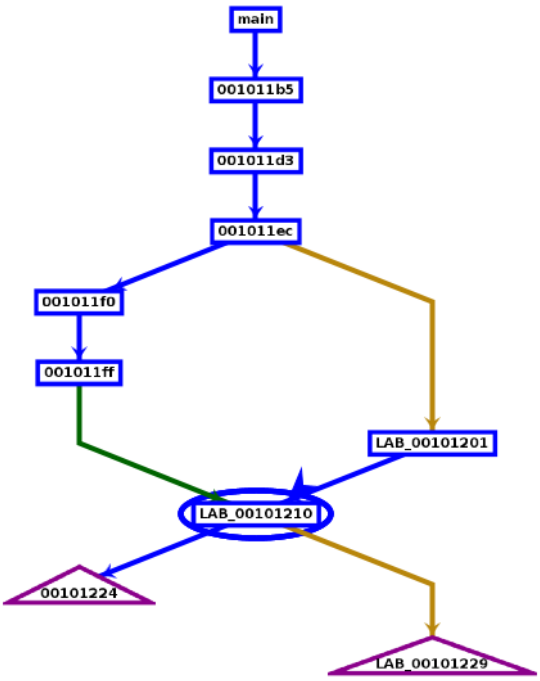


Image 37. Graph visualization.

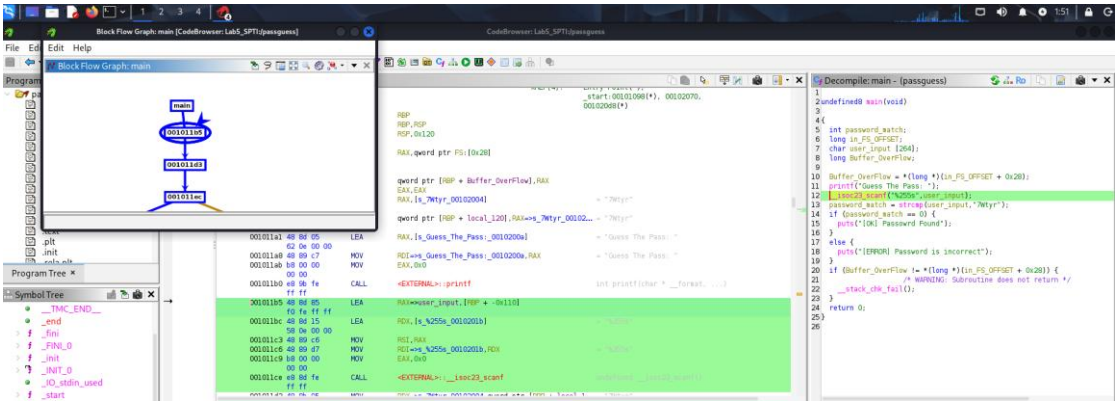


Image 38. Input validation by STRCMP with the literal constant '7Wtyr'.

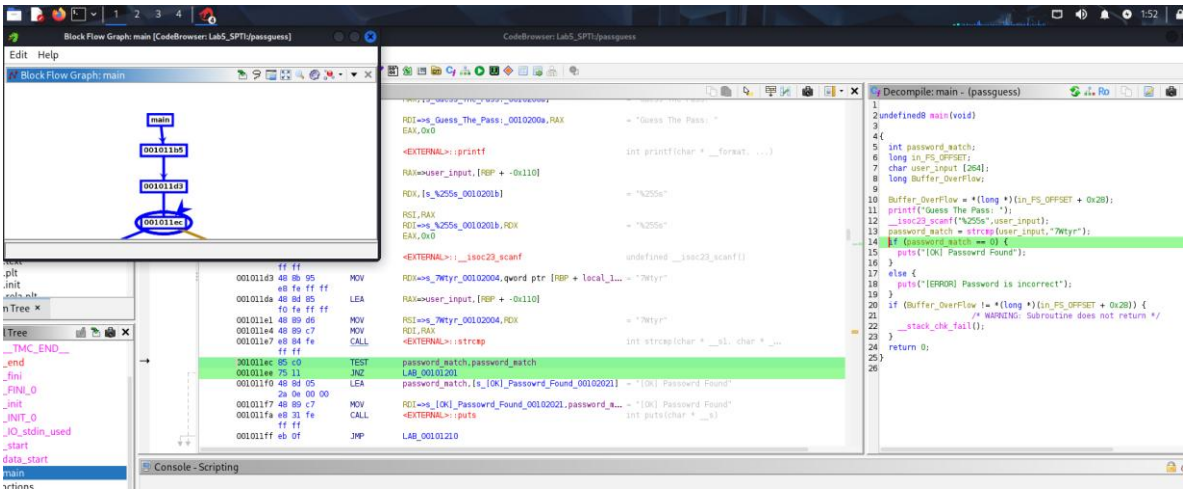


Image 39. An if conditional structure that determines the flow to access granted or denied after checking the input buffer.

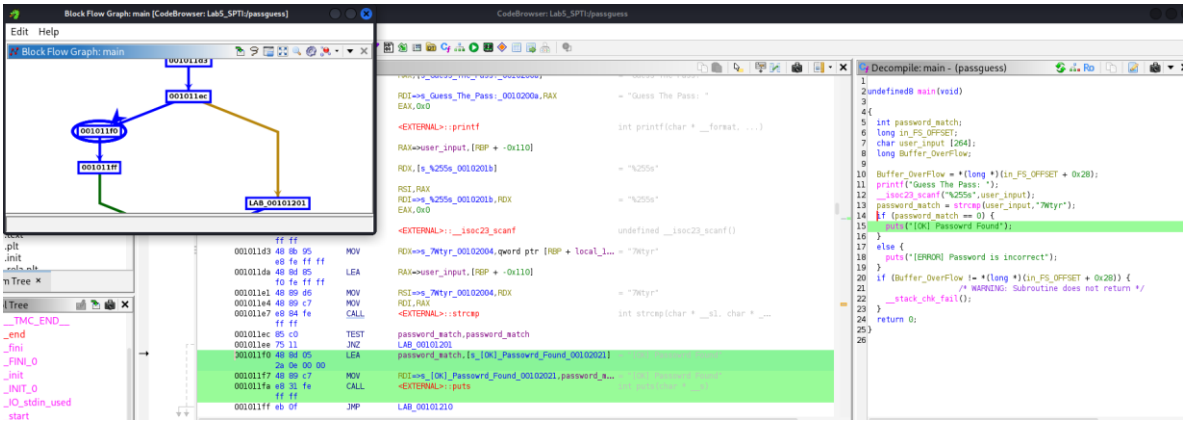


Image 40. comparison of user input with static password "7Wtyr" and security conditional branch.

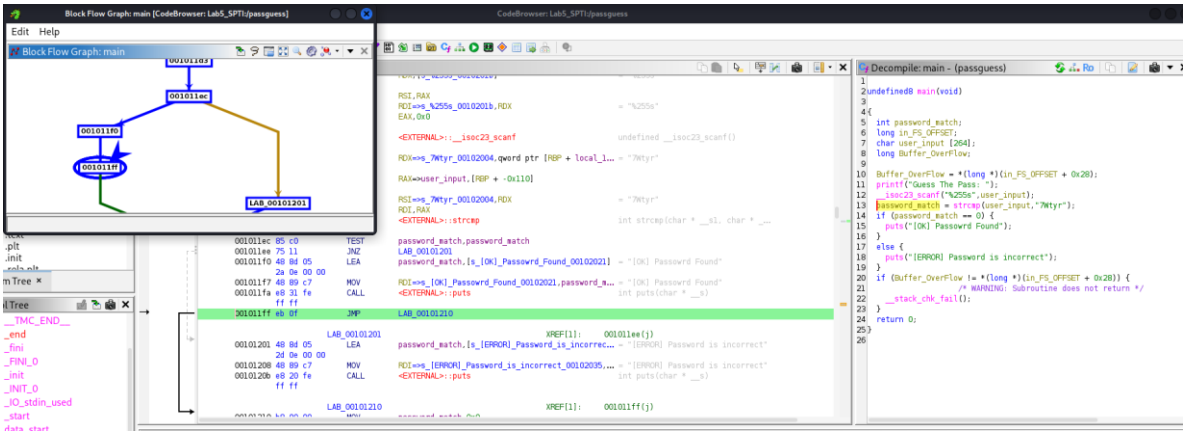


Image 41. Comparing User Input to Static Password

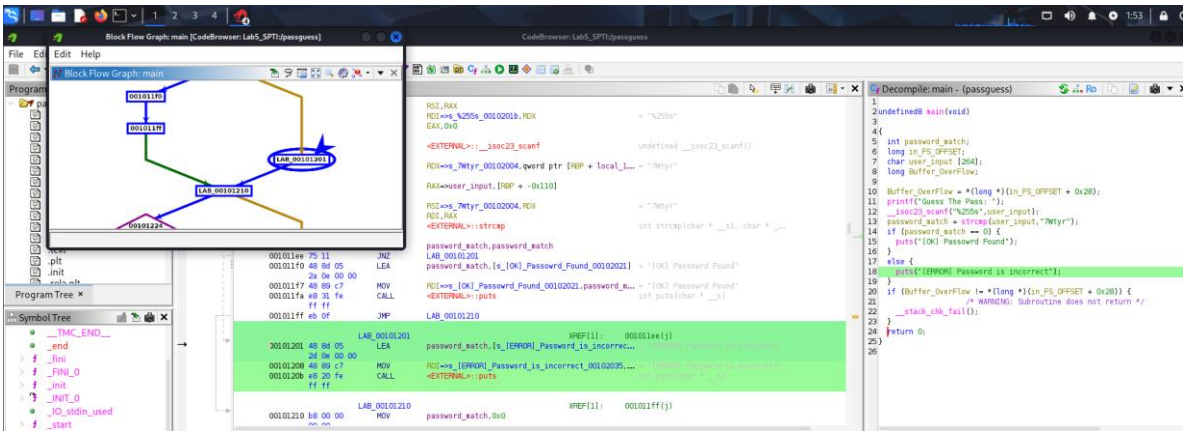


Image 42. Branching in case the password does not match, throws the error "Password is incorrect"

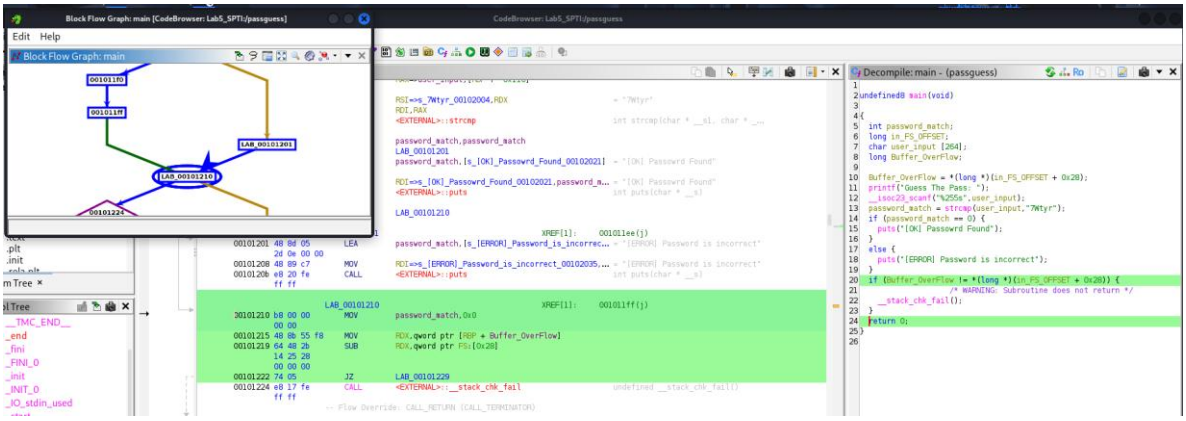


Image 43. comparison of user input with the constant "7Wtyr" and decision logic for system access.

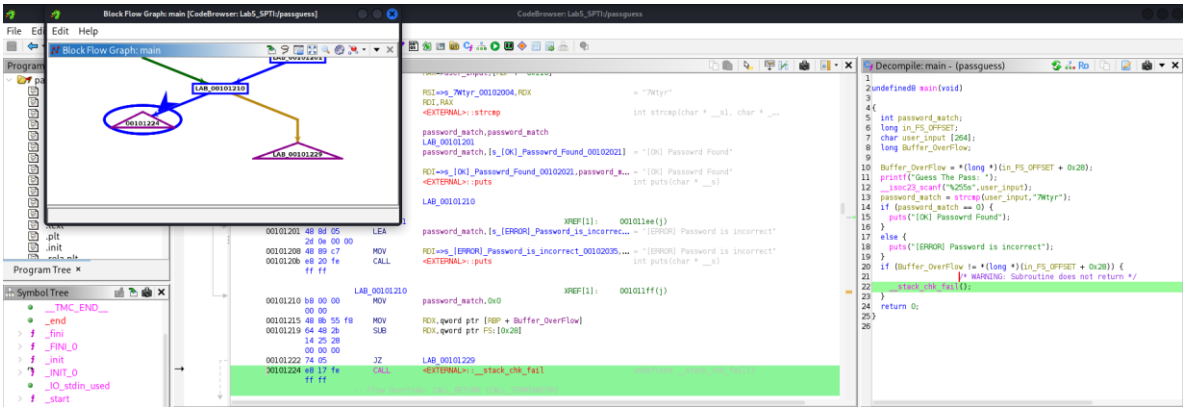


Image 44. Answer in case you have different passwords.

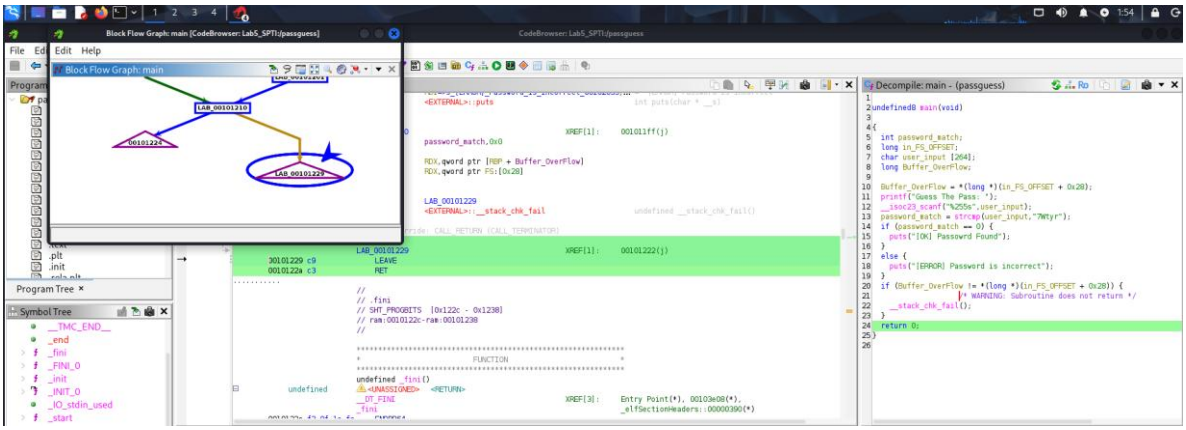


Image 45. Answer in case you have the same passwords.

Conclusions:

Ivan Santiago Forero Torres

- Through this lab, we were able to verify that the security of a binary does not lie only in its compilation. By using static analysis tools such as strings and Ghidra, we identified that the program stores the password '7Wtyr' in plain text within the data section. This finding underscores a critical vulnerability: any user with access to the executable can extract sensitive credentials without needing to run it, demonstrating the importance of implementing obfuscation techniques or *hashing* processes to protect sensitive data in software

Juan Sebastián Beltran Rodríguez

- From a development perspective, it was fascinating to observe how high-level code in C translates into low-level logical structures. By using the Function Graph (CFG), we accurately visualize the decision point at address 001011ec, where the jne instruction acts as the program's 'traffic light'. We understood that the flow to the success or error message depends exclusively on the value in the RAX register after the execution of strcmp. This visual analysis made it easier to understand if/else branching logic that would otherwise be difficult to follow in a pure assembler dump.

Nicolas Cuervo Naranjo

- The use of GDB was instrumental in dynamically validating our hypotheses. Thanks to the stepi and finish commands, we were able to intercept the exact moment when the user's input was compared to the actual key in the RDI and RSI records. Observing the state change in the registers and how the return value 0x31 (49) caused the jump to the error block allowed us to understand the lifecycle of an instruction in the processor. This hands-on experience with log-level debugging is invaluable for diagnosing security flaws and understanding the actual behavior of hardware versus software.